LINUX INTERVIEW PREPARATION GUIDE

CHAPTER

13

Memory Management and Performance Troubleshooting

Interview-focused notes for DevOps, SysAdmin and Cloud Engineers (1–5 years of experience)

IN THIS CHAPTER

01 Introduction
02 Linux Memory Management
03 How Linux Uses RAM
04 Swap Memory
05 Memory Statistics
06 The OOM Killer
07 CPU Performance
08 Load Average
09 Disk I/O Performance
10 Reading vmstat
11 Memory Leaks
12 Troubleshooting Methodology
13 Production Scenarios
14 Performance Cheat Sheet
15 Best Practices
16 Interview Q&A
17 Takeaways and Exercises

Ravi Tomar

Linux · DevOps · Cloud Infrastructure

Part 1 · Core Linux

github.com/rv-ops/linux-interview-preparation-guide

CHAPTER OBJECTIVE

Understand how Linux manages physical memory, virtual memory, swap and the page cache, how the OOM killer decides what to terminate, and how to read CPU, load average and disk I/O metrics. Learn a structured method for troubleshooting slow or unresponsive production servers. Chapter 12 introduced the monitoring commands; this chapter explains the concepts behind them and how to diagnose problems.

1 Introduction

Performance troubleshooting is an important responsibility of a Linux and DevOps engineer. A production server may suffer from:

- An application becoming slow or unresponsive.
- CPU utilization reaching 100%.
- RAM usage increasing continuously.
- The server running out of memory.
- Processes getting killed unexpectedly.
- Slow disk operations.
- High load average.

To troubleshoot these effectively, you must understand how Linux manages memory, CPU, swap and disk I/O. This chapter focuses on the concepts behind system performance, identifying bottlenecks and investigating real incidents.

2 Understanding Linux Memory Management

Linux uses RAM to store running programs, application data, kernel data and filesystem cache. When an application starts, the kernel allocates memory to it. As applications run, their memory needs grow and shrink, and Linux manages available memory dynamically.

Concept	Explanation
Physical memory	Actual RAM installed in the server
Virtual memory	The address space presented to each process
Page	Fixed-size unit of memory management (commonly 4 KB)
Page cache	RAM used to cache filesystem data
Swap	Disk space used as an extension of memory
OOM killer	Kernel mechanism that terminates processes under severe memory pressure

Physical Memory vs Virtual Memory

Physical memory (RAM) is the actual hardware memory. **Virtual memory** gives each process its own virtual address space. The kernel maps virtual addresses to physical memory pages or other backing storage.

For example, two different processes can use the same virtual address without touching the same physical memory. Virtual memory provides process isolation, memory protection, efficient allocation, shared memory and memory-mapped files.

> **INTERVIEW NOTE**
>
> Virtual memory is not just another name for RAM. It is an abstraction that lets each process use its own address space independently of the physical memory layout.

3 How Linux Uses RAM

Linux does not leave unused RAM idle. It uses free memory for caching to improve performance. When you read a file, Linux may keep its contents in the page cache, so a second read is served from RAM instead of disk. This is why a healthy Linux server can still show low free memory.

The `free -h` fields were explained in Chapter 12. The key point here is which value to trust.

Which Memory Value Should You Focus On?

For normal checks, focus on `available`, not `free`. A server with only 1 GB free is fine if it has several GB available, because Linux can reclaim cache when applications need it.

PRODUCTION EXAMPLE

A 16 GB server shows: used 7 GB, free 1 GB, available 8 GB. This is **not** memory pressure. Most of the "used" memory is reclaimable cache, and 8 GB is available for new applications.

4 Understanding Swap Memory

Swap is disk space used to hold memory pages that are not currently needed in RAM. When memory pressure rises, the kernel can move inactive pages to swap. Swap can be a dedicated partition or a swap file (covered in Chapter 6).

Swap provides extra capacity, handles temporary spikes and reduces the chance of an immediate out-of-memory condition. But it is far slower than RAM, and heavy swapping causes severe performance degradation.

Check Swap

```
# Current usage
free -h
swapon --show
cat /proc/swaps

# Swap activity: si (swap in) and so (swap out) per second
vmstat 1
```

If `si` and `so` show continuous significant activity, the server is likely under memory pressure. Brief spikes are normal.

Swappiness

Swappiness controls how readily the kernel swaps out pages (0 to 100).

```
# Current value
cat /proc/sys/vm/swappiness

# Change temporarily
sudo sysctl vm.swappiness=10

# Make it permanent
echo "vm.swappiness=10" | sudo tee /etc/sysctl.d/99-swappiness.conf
sudo sysctl --system
```

> **IMPORTANT**
>
> A lower swappiness value does **not** disable swap. Do not change it blindly; the right value depends on the workload. See Chapter 6 for swappiness and Chapter 8 for sysctl basics.

5 Understanding Memory Statistics

Linux exposes detailed memory data through `/proc`.

```
cat /proc/meminfo
```

Field	Meaning
`MemTotal`	Total physical RAM
`MemFree`	Completely unused RAM
`MemAvailable`	Estimated memory available for applications
`Buffers` / `Cached`	Block-device buffers and filesystem cache
`SwapTotal` / `SwapFree`	Total and available swap
`Slab`	Kernel memory used for caches of kernel objects

Per-Process Memory

```
# Top memory consumers
ps aux --sort=-%mem | head -10

# One process (replace 1234 with the real PID)
ps -p 1234 -o pid,comm,%mem,rss,vsz

# Detailed kernel view of a process
cat /proc/1234/status   # VmRSS, VmSwap, VmPeak, etc.
```

Field	Meaning
`RSS` (VmRSS)	Resident Set Size: physical RAM the process is actually using
`VSZ` (VmSize)	Virtual memory size: total address space, much of which may not be in RAM

INTERVIEW TRAP

RSS and VSZ are not interchangeable. A large VSZ is normal and does not mean the process is using that much RAM. Judge real memory use by RSS. Shared libraries are also counted in each process's RSS, so summing RSS across processes overcounts shared memory.

6 Understanding the OOM Killer

When Linux cannot satisfy memory demand even after reclaiming cache, the kernel invokes the Out Of Memory (OOM) killer, which terminates one or more processes to recover memory. Common causes include memory leaks, too many processes, oversized workloads and container memory limits.

Check OOM Events

```
journalctl -k | grep -i "out of memory"
dmesg -T | grep -i "killed process"
```

```
Out of memory: Killed process 2450 (java) total-vm:...
```

OOM Score

The kernel gives each process an OOM score; a higher score makes it a more likely victim. The score is influenced by how much memory the process uses and by its adjustment value.

```
# Current score and the admin-tunable adjustment (-1000 to 1000)
cat /proc/1234/oom_score
cat /proc/1234/oom_score_adj
```

WARNING

Do not blindly protect an application from the OOM killer by lowering its `oom_score_adj`. If the real cause is a memory leak, protecting that process can push the kernel to kill something more important and leave the server unable to recover.

Example **A Java application suddenly stopped**

```
systemctl status application
journalctl -k | grep -i "killed process"
journalctl -u application -n 100
free -h
```

If the OOM killer terminated it, investigate: a memory leak, an oversized or missing heap limit, insufficient server RAM, other memory-hungry processes, or a container memory limit that is too low.

7 Understanding CPU Performance

The CPU executes application work. Sustained high CPU usage causes slow responses, delayed jobs and higher latency.

```
# CPU details
lscpu

# Live view
top

# Per-core detail every second (sysstat package)
mpstat -P ALL 1
```

Key CPU Metrics

Metric	Meaning
`%user`	CPU time running user-space processes
`%system`	CPU time running kernel operations
`%idle`	CPU idle time
`%iowait`	CPU time waiting for I/O to complete
`%steal`	Time a virtual CPU waited while the hypervisor ran other workloads

```
%Cpu(s): 70.0 us, 15.0 sy, 0.0 ni, 10.0 id, 5.0 wa
```

Here the system is busy: 70% user, 15% system, only 10% idle, 5% I/O wait.

INTERVIEW TRAP

High `%steal` on a cloud VM means the hypervisor is giving your CPU time to other tenants (a "noisy neighbour" or an exhausted burstable instance). The fix is usually a larger or different instance type, not changes inside the OS.

Example **CPU usage above 90%**

1. Check overall CPU and which processes are responsible (`top`, `ps aux --sort=-%cpu | head`).
2. Check whether it started after a recent deployment.
3. Inspect application logs and threads.
4. Check whether traffic increased.

Do not restart the server first. Identify the source of CPU consumption.

8 Understanding Load Average

Load average is the average number of tasks that are running, waiting for CPU, or waiting in uninterruptible sleep (usually disk I/O). Chapter 12 introduced it; here is how to interpret it for troubleshooting.

```
uptime
nproc
```

```
load average: 4.20, 3.80, 2.50   (1, 5 and 15 minute averages)
```

Always compare the load with the CPU count from `nproc`. On a 4-CPU server, a load of 4 means fully busy; a load of 12 means many tasks are waiting. The 1, 5 and 15 minute values show the trend: rising or falling.

INTERVIEW TRAP

A high load average does **not** always mean high CPU usage. Tasks stuck in uninterruptible sleep (D state) waiting for disk raise the load even while the CPU is mostly idle. Always confirm with CPU and I/O metrics before concluding it is a CPU problem.

9 Understanding Disk I/O Performance

Even with healthy CPU and memory, slow storage can make applications slow. Causes include slow devices, heavy database reads/writes, excessive logging, full disks, deep I/O queues and network-storage latency.

```
# Extended statistics every second
iostat -xz 1

# Which processes are doing the I/O (may need installing)
sudo iotop

# Capacity (see Chapter 6)
df -h
```

Field	Meaning
`r/s` , `w/s`	Read and write operations per second
`rkB/s` , `wkB/s`	Read and write throughput
`r_await` , `w_await`	Average time per request, including queue time
`aqu-sz`	Average I/O queue length
`%util`	How busy the device is

PRODUCTION EXAMPLE

A database server is slow, but CPU is only 25% and memory is healthy. `iostat` shows high `await` and `%util` near 100%. This points to a storage bottleneck, not CPU. Investigate heavy queries, storage performance and I/O patterns.

10 Reading vmstat

`vmstat` shows processes, memory, swap, I/O and CPU together, which makes it good for a quick overall picture.

```
vmstat 1
```

```
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa
 2  0      0 500000  30000 900000    0    0    10    20  100  200 20  5 70  5
```

Column	Meaning
`r` / `b`	Processes waiting to run / blocked on I/O
`si` / `so`	Swap in / out per second (non-zero = active swapping)
`bi` / `bo`	Blocks read from / written to devices
`us` / `sy` / `id` / `wa`	CPU time: user / system / idle / I/O wait

INTERVIEW TRAP

The **first** vmstat line is an average since boot, so ignore it. Read from the second line onward for the live interval. A high `b` count with high `wa` points to an I/O bottleneck; sustained `si` / `so` points to memory pressure.

11 Understanding Memory Leaks

A memory leak happens when an application allocates memory but never releases it, so its usage grows over time, for example from 500 MB to 1 GB to 4 GB, until it becomes unstable or is killed. Rising memory alone does not prove a leak; it may just be a growing workload.

How to Investigate

```
# Identify the growing process and watch it over time
ps aux --sort=-%mem | head
top

# Inspect its memory
cat /proc/1234/status

# Application logs
journalctl -u application
```

For Java, also look at JVM heap usage, garbage-collection logs, heap dumps and profiling. For containers, check the container's memory usage and configured limits.

DEVOPS POINT

Adding RAM only delays the symptom of a leak.

The real fix is in the application code. More memory just makes the crash happen later.

12 Performance Troubleshooting Methodology

When a server is slow, follow a structured path instead of restarting services at random. This expands the "golden order" (CPU, memory, disk, network, logs) with the reasoning behind each step.

Step 1 Overall health

```
uptime        # load trend vs nproc
free -h       # available memory and swap
df -h         # any full filesystem
```

Step 2 CPU

```
top
ps aux --sort=-%cpu | head
```

Is the CPU saturated, and which process is responsible?

Step 3 Memory and swap

```
ps aux --sort=-%mem | head
vmstat 1
```

Look for memory pressure, unusual growth and continuous swap in/out.

Step 4 Disk I/O

```
iostat -xz 1
```

Look for high `await` and `%util`.

Step 5 Logs and recent changes

```
journalctl -p err -b
journalctl -k -b
journalctl -u application -n 100
```

Correlate the problem with recent deployments, config changes, traffic increases or new jobs. Then apply the right fix (fix a leak, optimize code, resolve storage latency, roll back a deployment, or add capacity only when justified), and verify recovery afterwards.

13 Real-World Production Scenarios

Scenario 1 Server at 100% CPU

Symptoms: slow website, slow SSH, high CPU.

```
uptime
top
ps aux --sort=-%cpu | head
```

Likely causes: infinite loops, traffic spike, expensive queries, background jobs. **Fix:** identify the process, understand its workload and correct the root cause.

Scenario 2 Application killed unexpectedly

Symptoms: the process disappears with no clean shutdown in its logs.

```
journalctl -k | grep -i "killed process"
free -h
ps aux --sort=-%mem | head
```

Likely cause: the OOM killer. **Fix:** investigate memory usage, heap settings, leaks, other processes and capacity.

Scenario 3 Slow despite low CPU

```
iostat -xz 1
vmstat 1
ss -s
```

Likely causes: slow disk, database latency, network issues, external API delays, processes blocked on I/O. **Fix:** find which dependency is causing the wait.

Scenario 4 Memory usage keeps rising

```
free -h
ps aux --sort=-%mem | head
```

Likely cause: a memory leak or growing workload. **Fix:** analyze the application's memory behavior and fix the source of allocation.

Scenario 5 High load average

```
uptime
nproc
top
vmstat 1
iostat -xz 1
```

Fix: decide whether the bottleneck is CPU, storage or too many runnable tasks before acting.

Scenario 6 Container keeps restarting (OOMKilled)

```
# Docker
docker ps -a
docker logs <container>
docker stats

# Kubernetes: look for Reason: OOMKilled in the events
kubectl describe pod <pod>
kubectl logs <pod> --previous
```

A container is killed when it exceeds its memory limit, even if the node has free RAM. **Fix:** reduce application memory use, or set appropriate memory requests and limits. Containers are covered further in Chapter 15.

14 Performance Troubleshooting Cheat Sheet

Requirement	Command
Available memory	`free -h`
Detailed memory info	`cat /proc/meminfo`
Check swap	`swapon --show`
Memory, swap and CPU together	`vmstat 1`
Top memory processes	`ps aux --sort=-%mem | head`
Top CPU processes	`ps aux --sort=-%cpu | head`
CPU info and count	`lscpu` , `nproc`
Load average	`uptime`
Disk I/O	`iostat -xz 1`
I/O per process	`iotop`
OOM events	`journalctl -k | grep -i "out of memory"`
Process memory detail	`cat /proc/PID/status`

15 Best Practices

1. **Judge memory by** `available` , not free RAM alone.
2. **Watch swap activity:** continuous swapping signals memory pressure.
3. **Investigate OOM events** to find why a process was killed.
4. **Distinguish spikes from saturation:** a short CPU spike is not sustained load.
5. **Interpret load average** against CPU count and I/O.
6. **Monitor disk latency:** storage can be the bottleneck even at low CPU.
7. **Compare with historical metrics** to know what is normal.
8. **Collect evidence before restarting:** logs and metrics vanish after a restart.
9. **Set appropriate container limits** for memory and CPU.
10. **Fix root causes:** extra RAM or CPU rarely fixes an inefficient application.

16 Common Interview Questions

Q1. What is virtual memory in Linux?

It gives each process its own virtual address space, which the kernel maps to physical memory and other backing storage. This provides isolation and efficient memory management.

Q2. What is the difference between RAM and swap?

RAM is fast physical memory used directly. Swap uses disk to hold pages not currently needed in RAM and is much slower.

Q3. Why does Linux show low free memory?

It uses free RAM for cache and buffers, which it can reclaim when needed. Use the `available` field to judge real availability.

Q4. What is the OOM killer?

A kernel mechanism that terminates selected processes when the system cannot satisfy memory demand even after reclaiming memory.

Q5. How do you find the process using the most memory?

```
ps aux --sort=-%mem | head
```

Q6. What is swappiness?

A kernel setting (0 to 100) that controls how readily pages are moved to swap. A lower value does not disable swap.

Q7. Does a high load average always mean high CPU usage?

No. Tasks waiting on disk or other uninterruptible operations also raise the load average.

Q8. What is the difference between RSS and VSZ?

RSS is the physical RAM a process is using now. VSZ is its total virtual address space, much of which may not be in RAM.

Q9. What does `%iowait` indicate?

The percentage of CPU time spent waiting for I/O to complete. High values suggest a storage bottleneck, but confirm with `iostat`.

Q10. What does high `%steal` mean on a cloud VM?

The hypervisor is giving your vCPU time to other tenants, or a burstable instance has run out of credits. Usually fixed by resizing the instance, not inside the OS.

Q11. What is a memory leak, and does more RAM fix it?

It is memory that an application allocates but never frees. Adding RAM only delays the crash; the fix is in the application.

Q12. How do you investigate a process killed by the OOM killer?

```
journalctl -k | grep -i "killed process"
```

Then inspect memory usage, application logs, runtime settings and limits.

Q13. How do you troubleshoot a server that is slow but has low CPU?

Check memory and swap (`free`, `vmstat`), disk latency (`iostat`) and connections (`ss`). The cause is often disk I/O, a database, an external API or the network.

Q14. Why ignore the first line of `vmstat`?

It shows averages since boot. Read from the second line for the live interval.

Q15. Why is a container OOMKilled even when the node has free memory?

The container exceeded its own memory limit. The limit, not total node memory, triggers the kill.

17 Chapter Takeaways

After completing this chapter, you should understand:

- ✓ How Linux manages physical and virtual memory.
- ✓ Why Linux uses RAM for caching and why `available` matters.
- ✓ How swap works and when it becomes a problem.

- ✓ How the OOM killer and OOM score work.
- ✓ The difference between RSS and VSZ.
- ✓ How to read CPU metrics, including `%iowait` and `%steal`.
- ✓ How to interpret load average against CPU count.
- ✓ How to identify disk I/O bottlenecks.
- ✓ How to read `vmstat` correctly.
- ✓ How to investigate memory leaks.
- ✓ A structured method for troubleshooting slow servers.
- ✓ How to troubleshoot Docker and Kubernetes memory issues.

PRACTICAL EXERCISES

Perform these on a Linux test machine.

1. Check total, used and available RAM, and explain the difference.
2. Display detailed memory information from `/proc/meminfo`.
3. Check whether swap is enabled and monitor `si` / `so` with `vmstat`.
4. Identify the top five processes by memory and by CPU.
5. Compare RSS and VSZ for one process.
6. Check the load average and compare it with `nproc`.
7. Monitor disk I/O with `iostat -xz 1` and identify the busiest device.
8. Inspect kernel logs for OOM killer events.
9. Read `/proc/PID/status` for a running process.
10. Run `vmstat 1` and interpret the live interval, ignoring the first line.
11. Run `docker stats` and observe container memory usage.
12. Inspect a Kubernetes pod for an `OOMKilled` event using `kubectl describe pod`.

INTERVIEW TARGET

Be able to explain RAM, virtual memory, swap, the OOM killer, CPU metrics, load average and disk I/O, and describe a systematic approach to troubleshooting a slow production server.

NEXT CHAPTER

Chapter 14: Linux Internals and Kernel Fundamentals

You will learn how applications interact with the kernel through system calls, how processes and threads are created and scheduled, and how to use /proc, sysctl, lsof and strace to investigate low-level problems.

END OF CHAPTER 13